

Progress Report: LLM Hallucination Detection via Uncertainty

Mustafacan Koc
SEDS 537 – Machine Learning

May 2026

1 Project Title and Problem Definition

The project title is *LLM Hallucination Detection via Uncertainty*. The problem is to detect whether a large language model answer is supported or hallucinated. This is formulated as a binary classification task.

The input to the system is a question, an optional supporting context, and a candidate answer. The output is a label where 0 means supported/truthful and 1 means hallucinated/unsupported. This problem is important because fluent but unsupported answers reduce the reliability of LLM-based systems in factual tasks.

The main challenge is that hallucinated answers can still be fluent and plausible. Therefore, the project uses uncertainty signals from an open-source LLM instead of relying only on surface text patterns.

2 Dataset and Preprocessing Status

Two datasets are used in the project. HaluEval QA is the main dataset for training and initial evaluation. TruthfulQA generation is prepared for external evaluation in the next phase.

The processed HaluEval QA file contains 20,000 examples. Each original question has one supported answer and one hallucinated answer. The processed TruthfulQA file contains 5,918 examples created from correct and incorrect answer candidates.

Both datasets were converted into a shared JSONL schema with the columns `id`, `dataset`, `task`, `prompt`, `context`, `answer`, `label`, `source`, and `metadata`. The `prompt` stores the question, `context` stores the supporting evidence when available, and `answer` stores the candidate answer to be classified. The `label` column is the target variable.

For example, in HaluEval QA, the same question may produce two processed rows: one row with the supported answer “*Arthur’s Magazine*” labeled as 0, and one row with the hallucinated answer “*First for Women was started first*” labeled as 1. In TruthfulQA, correct answer candidates are labeled as 0 and incorrect answer candidates are labeled as 1.

For HaluEval, the raw QA data was downloaded and converted into supported and hallucinated labeled examples. For TruthfulQA, correct answers were labeled as 0 and incorrect answers were labeled as 1. The data preparation script is implemented in `scripts/prepare_data.sh`.

For the first experiment, HaluEval was split using a group-aware 80/20 train-test split. The split keeps paired supported and hallucinated answers from the same question in the same partition. This avoids direct leakage between train and test examples.

3 Literature Review Progress

The literature review has focused on hallucination benchmarks, uncertainty-based detection, semantic uncertainty, and evidence-aware hallucination evaluation. The most relevant papers reviewed so far are summarized below.

HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models is the most important dataset paper for the current implementation. It introduces labeled hallucination examples across several task types, and the QA portion is used as the main dataset in this project. This paper supports the label definition and the choice of HaluEval as the first controlled benchmark.

Detecting Hallucinations in Large Language Models Using Semantic Entropy is central to the uncertainty direction of the project. The paper argues that uncertainty should sometimes be measured over meanings rather than only surface token forms. It motivates the use of entropy and self-consistency ideas, although the current implementation starts with token-level entropy.

Enhancing Uncertainty-Based Hallucination Detection with Stronger Focus is closely related to the implemented feature extraction stage. It studies how token probability signals can be improved by focusing on more informative tokens. This supports the use of token confidence, log-probability, low-confidence ratios, and related summary features.

Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs is relevant because semantic entropy can be computationally expensive when it requires multiple generations. The paper explores cheaper ways to approximate semantic uncertainty. This is useful for future work if the project extends beyond the current single-pass token uncertainty pipeline.

ChainPoll: A High Efficacy Method for LLM Hallucination Detection represents a different direction based on LLM-as-judge style hallucination detection. It is useful because it contrasts uncertainty-only detection with methods that explicitly reason about answer correctness and adherence. This helps position the current project as a lightweight uncertainty-based approach.

Hallucination Detection in Large Language Models via Multi-Granular Uncertainty Quantification is very close to the project idea. It combines multiple uncertainty signals such as token-level, sequence-level, and distributional uncertainty. It supports the planned ablation study, where confidence/log-probability features and entropy features will be compared separately.

A Probabilistic Framework for LLM Hallucination Detection via Belief Tree Propagation is relevant to the longer-term multi-signal direction. It models relationships between related claims and uses probabilistic inference to reduce noisy hallucination judgments. This supports the idea that a single uncertainty score may be useful but incomplete.

Overall, the literature supports the current technical direction: hallucination detection can be framed as supervised binary classification, and uncertainty signals such as token probability, log-probability, entropy, and consistency are meaningful features for this task.

4 Baseline Models

Three baseline classifiers have been selected and implemented:

- Logistic Regression
- Linear Support Vector Machine (SVM)
- Random Forest

These models are suitable because the extracted uncertainty representation is a compact tabular feature set. Logistic Regression provides an interpretable linear baseline. Linear SVM provides a margin-based linear classifier. Random Forest provides a non-linear ensemble baseline.

The model implementations are placed under `src/models/`. The evaluation pipeline is implemented in `src/evaluation/evaluate_uncertainty_classifiers.py`.

5 Initial Experimental Results

The current experiment uses Qwen2.5-3B as a feature extractor. The model is not asked to judge hallucination directly. Instead, it scores the provided answer token by token, and the project computes uncertainty features from the model’s probability distribution.

The current feature table includes 15 independent variables, including token probability, log-probability, low-confidence ratio, entropy, and high-entropy ratio features. The dependent variable is the binary hallucination label.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9875	0.9865	0.9885	0.9875	0.9986
Linear SVM	0.9885	0.9890	0.9880	0.9885	0.9986
Random Forest	0.9883	0.9856	0.9910	0.9883	0.9988

Table 1: Initial HaluEval QA results using a grouped 80/20 train-test split.

The initial results are strong on HaluEval QA. However, these results should be interpreted carefully. They show that uncertainty and entropy features separate the HaluEval supported and hallucinated examples well, but they do not yet prove general hallucination detection performance across datasets. External evaluation on TruthfulQA is needed to test generalization.

6 Planned Improvements and Technical Direction

The next technical step is to extract the same uncertainty and entropy features for TruthfulQA and evaluate whether models trained on HaluEval transfer to TruthfulQA. This will help determine whether the high HaluEval scores reflect true generalization or are partly due to the structure of HaluEval.

Another planned improvement is to test a different feature-extraction LLM, possibly a smaller open-source model. This will show whether the uncertainty features are stable across LLMs or strongly dependent on Qwen2.5-3B.

If time allows, the project may also add self-consistency features and retrieval/evidence-agreement features, as described in the original proposal.

7 Ablation and Error Analysis Plan

The planned ablation study will compare different feature groups:

- confidence and log-probability features only
- entropy features only
- all uncertainty features together

This will show whether entropy adds useful information beyond token confidence and log-probability. For error analysis, I plan to inspect false positives and false negatives from the classifier predictions. The analysis will focus on cases where the model is overconfident but wrong, and on examples where supported and hallucinated answers have similar uncertainty patterns.

8 Visualization and Interpretation Plan

The final report will include visualizations that explain the model behavior rather than only decorating the results. Planned visualizations include confusion matrices, ROC curves, and feature importance plots from the Random Forest model. Logistic Regression coefficients may also be inspected to understand which uncertainty features are most influential.

These visualizations have not been added yet, but the output prediction and metrics files are already available for producing them.

9 GitHub and Reproducibility Status

The GitHub repository is available at:

<https://github.com/cankoc35/SEDS537-MachineLearningProject>

The repository contains data preparation, feature extraction, model training, and evaluation scripts. The main reproducibility files are:

- `requirements.txt`
- `scripts/prepare_data.sh`
- `scripts/evaluate.sh`
- `README.md`

The current experiment can be reproduced by preparing the data, extracting Qwen uncertainty features, and running `scripts/evaluate.sh`. The README has been updated to describe the current project structure and commands.

10 Current Challenges and Next Steps

The main current challenge is interpreting the very high HaluEval results. They are promising, but external validation is needed before making broader claims. TruthfulQA is expected to be more challenging and will be used to test generalization.

The next steps are:

1. extract uncertainty and entropy features for TruthfulQA,
2. evaluate HaluEval-trained classifiers on TruthfulQA,
3. run the ablation study,
4. perform error analysis,
5. add final visualizations and discussion for the final report.